ConsensusScope: Graph-Based Safety Routing for Teacher-in-the-Loop AI Writing Feedback

You Tingrui and Li Jimei and Zhang Na and Xiong Ling
School of Information Science, Beijing Language and Culture University
yyting2006@163.com, Ljm@blcu.edu.cn
zn17333099816@163.com, lingdonghua99@163.com

Abstract

AI writing-feedback systems can produce fluent revision advice, but fluency is not the same as pedagogical safety: a suggestion may change a student's claim, introduce unsupported evidence, or overcorrect a draft. We present ConsensusScope, an interactive system for feedback-level safety routing in teacher-in-the-loop ESL writing support. The system treats each AI feedback item, rather than an essay score or final rewrite, as the object of decision. It normalizes feedback candidates into a shared schema, extracts deploy-time safety signals, constructs an auditable Feedback Safety Graph, and routes each item to auto_accept, teacher_review, or needs_more_evidence. A Streamlit/FastAPI demo supports single-essay review, batch review, teacher queueing, graph inspection, and report export. We evaluate the routing behavior with a stress suite, a public learner-corpus benchmark derived from JFLEG, CoNLL-2014, FCE, and W&I+LOCNESS, and a 30-item two-teacher diagnostic pilot. Across 349,782 generated feedback candidates, the default policy auto-routes 32.9% of candidates while sending all constructed incorrect or unsafe candidates to review. The teacher pilot shows 0.857 auto-accept precision against teacher-safe items and 0.694 within-one-point teacher agreement. These results support controlled, graph-backed safety-routing behavior, not classroom learning outcomes or real-world LLM feedback quality.

1 Introduction

AI writing feedback is useful only when teachers can control what reaches a student. Existing educational NLP work includes grammatical error correction (GEC) shared tasks (Ng et al., 2014; Bryant et al., 2019), learner-writing corpora such as FCE and JFLEG (Yannakoudakis et al., 2011; Napoles et al., 2017), and writing-support systems that help students revise drafts (Madnani et al., 2018). Recent LLMs make feedback generation easier, but they also increase the need for review: a fluent comment may be locally grammatical while being pedagogically unsafe, unsupported, or too interventionist.

ConsensusScope addresses this gap as an algorithmic routing problem rather than an automatic essay-scoring problem. Given an ESL draft and a set of AI-style feedback candidates, the target output is an item-level route: auto_accept, teacher_review, or needs_more_evidence. The technical challenge is that a feedback item cannot be judged from its issue label alone. A grammar suggestion may be safe in one context but wrong after a modal verb; a vocabulary edit may be fluent but shift meaning; and a persuasive content suggestion may introduce evidence that the student never provided.

The core idea is to convert each feedback item into a Feedback Safety Graph before making it student-facing. Instead of reducing feedback to one confidence score, the graph records how the target span, AI suggestion, predicted issue type, evidence status, extracted safety signals, and final route support one another. This graph representation lets the system expose the reason for a route, not merely the route itself.

This framing is intentionally practical. It supports teachers who need to triage AI feedback before student release, educational NLP developers who need observable failure modes, and researchers studying reliability in human-in-the-loop feedback pipelines.

The demo contributes: (1) a feedback-level safety-routing formulation for AI-generated

ESL writing feedback; (2) a Feedback Safety Graph Routing algorithm that extracts deploy-time safety signals, maps them to explicit graph dimensions, and applies conservative route thresholds without using teacher labels at inference time; (3) a usable Streamlit/FastAPI system for single-essay review, batch review, teacher queueing, evaluation, and graph-backed report export; and (4) a reproducible evaluation package that combines public learner-corpus routing diagnostics with a two-teacher offline pilot.

2 Related Work

Learner writing and GEC. GEC benchmarks have provided standardized correction tasks and learner-writing data, including CoNLL-2014 (Ng et al., 2014), BEA-2019 W&I+LOCNESS (Bryant et al., 2019), FCE (Yannakoudakis et al., 2011), and JFLEG (Napoles et al., 2017). These datasets primarily evaluate correction quality. ConsensusScope uses their correction annotations only as offline evidence for a downstream routing question: whether risky or incorrect feedback candidates remain in a review queue.

Automated writing feedback. Automated writing systems can provide revision support, but classroom use depends on how learners and teachers interpret the feedback (Madnani et al., 2018; Ranalli, 2018). ConsensusScope follows this caution by avoiding a teacher-replacement claim and making AI feedback inspectable before it becomes student-facing.

LLM reliability and adjudication. Multi-output and judge-based methods can help select answers or evaluate generations (Wang et al., 2023; Zheng et al., 2023; Liu et al., 2023). However, consensus or judge confidence is not equivalent to educational safety. Our demo treats model agreement as one observable signal alongside issue type, evidence status, meaning-change warnings, unsupported-claim cues, tone cues, and parse errors.

3 Comparison with Existing Systems

ConsensusScope differs from existing writing-support and evaluation systems in its decision unit. GEC tools produce corrections, essay-scoring systems assign essay-level labels, and

ESL draft(s) → feedback candidates → unified schema → Feedback Safety Graph → auto-accept / teacher review → report export

Figure 1: ConsensusScope turns AI writing feedback into graph-backed, auditable teacher-review routes rather than final automatic rewrites.

LLM-as-judge workflows assess generated outputs. ConsensusScope instead routes each feedback item before it reaches a student, exposing graph evidence behind release or review decisions (Table 1).

4 Feedback Safety Graph Routing

ConsensusScope is organized around an item-level routing algorithm embedded in a teacher-facing review system (Figure 1). A teacher can paste a single anonymized ESL draft or upload a batch CSV. The system generates no-API AI-style feedback candidates or accepts live provider outputs in the same schema. Each feedback item is then screened by the Feedback Safety Graph Router and displayed as either a low-risk local edit or a teacher-review item.

Unified feedback schema. Every feedback candidate is represented with a stable item-level schema: feedback_item_id, essay_id, target span, surrounding context, AI suggestion, rationale, model source, predicted issue type, and optional model agreement. This schema lets deterministic demo reviewers and live OpenAI-compatible providers feed the same routing and reporting components.

Problem definition. Let $f = (x, c, u, t, e, a)$ denote a feedback item with target span x, local context c, AI suggestion u, predicted issue type t, evidence status e, and optional agreement signal a. The router estimates $R(f) = (r, s, G)$, where r is a release route, $s \in [0, 1]$ is a bounded risk score, and G is an auditable graph connecting the observable signals to the route. The deploy-time objective is conservative: release only items whose observable signals indicate low-risk local feedback, while sending ambiguous, unsupported, meaning-changing, or unsafe items to teacher review.

Algorithm. The procedure in Figure 2 summarizes the Feedback Safety Graph Router.

System type	Main object	Typical output	ConsensusScope distinction
GEC / revision tools	Sentence edit	Corrected text	Audits whether feedback should be student-facing
Essay scoring systems	Whole essay	Score or rubric label	Avoids grading; routes item-level AI feedback
LLM-as-judge workflows	Model output	Preference or quality score	Uses graph-backed deploy-time safety signals
Feedback dashboards	Review records	Tables or filters	Connects target span, suggestion, risk, and route

Table 1: ConsensusScope is positioned as feedback safety routing rather than automatic correction, essay scoring, or generic model judging.

Feedback Safety Graph Routing Procedure
Input: feedback item f, local context c, evidence status e, optional model agreement a
Output: route r, risk level ℓ, risk score s, graph G
1. Extract deploy-time signals from (f, c, e, a)
2. Map signals to risk reasons and safety dimensions
3. Compute bounded risk score s from additive safety evidence
4. If parsing fails, set r = needs_more_evidence
5. Else if $s \geq 0.72$, set ℓ = high and r = teacher_review
6. Else if $s \geq 0.32$, set ℓ = medium and r = teacher_review
7. Else set ℓ = low and r = auto_accept
8. Build graph G with target span, suggestion, evidence, active dimensions, and route

Figure 2: The routing algorithm uses deploy-time signals only. Teacher ratings and public-corpus gold corrections are reserved for offline evaluation.

The algorithm extracts deploy-time signals such as issue-type risk, parse failure, evidence conflict, model agreement, meaning-change cues, unsupported claims, whole-draft rewriting, tone warnings, teacher-dependent wording, and broad target spans. Each signal contributes to a bounded risk score $s \in [0, 1]$ and to human-readable risk reasons, which are projected into six graph dimensions: local edit, meaning preservation, content grounding, pedagogical tone, feedback specificity, and model agreement.

Graph construction. For each feedback item, ConsensusScope constructs a compact directed graph with the core path target span -> AI suggestion -> active safety dimension -> routing decision. Graph nodes represent the student target span, surrounding context, AI suggestion, optional rationale, predicted issue type, evidence signal, activated safety dimensions, and route. Edges capture relations such as contains, is_revised_by, activates, and justifies_route. The exported route includes graph dimensions, active signals, a path summary, and JSON node/edge records.

Routing outputs. For each feedback item, the system outputs a risk level, recommended action, risk reasons, risk score, graph path, node/edge records, and a short explanation. Low-risk grammar, spelling, punctuation, and vocabulary edits can be routed to auto_accept; medium- and high-risk items are routed to teacher review, more-evidence, or reject states.

5 System Implementation

The teacher-facing implementation exposes the routing algorithm as a working demo rather than a table-only analysis. The Streamlit interface has pages for Review Workspace, Single Essay Review, Batch Review, AI Feedback Comparison, Teacher Queue, Effectiveness Evaluation, Reports, Settings/Diagnostics, and Design Reference. Users can inspect routed items, open graph paths, prioritize review queues, and export Markdown reports. A FastAPI backend handles review-session creation, SQLite persistence, teacher decisions, audit logs, and report export.

6 Target Users and Use Cases

The primary users are writing teachers, educational NLP developers, and NLP researchers. Teachers inspect which AI suggestions require judgment before student release; developers debug feedback pipelines with normalized routing records; and researchers test review-routing policies and risk signals. The system is intended for human-review routing rather than fully automatic writing evaluation.

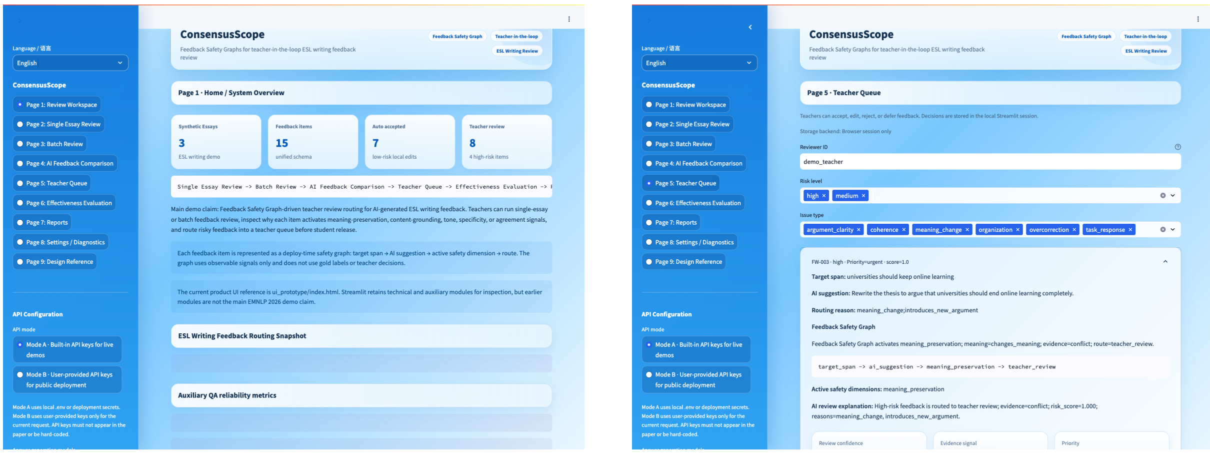

Figure 3: Teacher-facing demo screens. Left: the review workspace summarizes the safety-routing workflow. Right: the teacher queue prioritizes feedback items that need review before student release.

7 Evaluation

We evaluate the implemented routing algorithm, not a full writing-feedback generation model. The evaluation asks whether Feedback Safety Graph Routing separates safe local edits from feedback that is incorrect, meaning-changing, unsupported, or unsafe, and whether the graph signals exposed by the algorithm match teacher-facing diagnostic patterns. We use three types of evidence.

Algorithmic stress checks. The packaged demo contains 15 feedback items with expected routes and 16 stress-test cases for dangerous suggestions such as thesis reversal, whole-essay rewriting, unsupported factual claims, harsh tone, low agreement, and parse failures. The current router matches the expected action and risk labels on all 31 items. This is an algorithm sanity check rather than a classroom outcome study.

Public learner-corpus benchmark. We construct an offline routing benchmark from public learner-correction corpora. For each original/corrected sentence pair, we align source and reference tokens to create feedback-level gold edits. We then generate one gold-derived correct feedback candidate and one or more deliberately risky distractor candidates for each edit. The router sees only the deploy-time feedback fields; gold labels are used afterward to compute whether incorrect or unsafe candidates were routed to review. Table 2 reports the runs completed on directly downloadable public data.

Two-teacher diagnostic pilot. To test whether the review routes align with human judgments, we collected a small blind pilot from two English teachers over 30 anonymized AI feedback items. Each teacher rated correctness, meaning preservation, student readiness, usefulness, clarity, and direct-release suitability on 1–5 scales. The teacher ratings are used only as offline diagnostics. They are not visible to the deploy-time router. The pilot identified three borderline auto-release patterns that are observable at deploy time: teacher-dependent wording ("if the teacher wants"), semantic drift in vocabulary edits ("remember" to "learned"), and wrong local grammar corrections after modal verbs ("can make" to "makes"). We added conservative graph signals for these cases without using teacher labels during routing. On the 30-item pilot, the final router sends 76.7% of items to review, reaches 1.000 recall for teacher-marked review-needed items and teacher-marked unsafe items, and obtains 0.857 auto-accept precision against teacher-safe items (Table 3). The mean within-one-point agreement between the two teachers over six rating dimensions is 0.694.

Case analysis. Table 4 illustrates how the same routing algorithm handles safe local edits, meaning-changing suggestions, unsupported claims, and wrong local grammar corrections. The pilot did not make the router use teacher ratings at deploy time; it exposed which observable signals should be added to the rule set.

Source	Records	Gold edits	Candidates	Auto share	Review share
JFLEG	1,501	3,938	11,587	0.320	0.680
CoNLL-2014	2,624	5,110	15,126	0.324	0.676
FCE	33,236	45,969	136,934	0.329	0.671
W&I+LOCNESS	38,692	62,384	186,135	0.329	0.671
Total	76,053	117,401	349,782	0.328	0.672

Table 2: Public learner-corpus routing benchmark. CoNLL-2014 aggregates two annotator runs; FCE aggregates train/dev/test; W&I+LOCNESS aggregates train/dev. W&I+LOCNESS test, Lang-8, and NUCLE training data are not redistributed because required labels or licenses are not directly available.

Teacher-aligned diagnostic	Value
Feedback items	30
Rating rows	60
Auto share	0.233
Review share	0.767
Review-needed recall	1.000
Unsafe reviewed recall	1.000
Auto precision vs. teacher-safe	0.857
Within-one-point teacher agreement	0.694

Table 3: Two-teacher offline diagnostic pilot. Teacher ratings are post-hoc diagnostics and are not used during deploy-time routing.

Review-routing utility. Table 5 summarizes the pooled policy results over 349,782 feedback candidates. The default policy accepts only low-risk items and routes medium/high-risk items to review. It auto-routes 32.9% of candidates while preserving all constructed incorrect or unsafe candidates in the review queue. The 1.000 auto accuracy should be interpreted narrowly: correct candidates are derived from public gold corrections, and incorrect candidates are constructed risk distractors. The number validates routing behavior under controlled conditions; it is not a claim that real LLM feedback is always correct.

8 Demo Scenario

The two-and-a-half-minute demo follows a teacher workflow: open Review Workspace, run Single Essay Review on a packaged anonymized ESL draft, inspect Feedback Safety Graph paths for auto-accepted and teacher-review items, process a batch CSV, compare AI feedback, select a high-risk item in Teacher Queue, and export a Markdown report. The Effectiveness Evaluation page shows stress-test metrics and the public learner-corpus aggregate table.

9 Release and Reproducibility

ConsensusScope is released under the MIT License. The public repository, live demo, narrated demo video, and backend API documentation are available at https://github.com/yyting2006-ai/ConsensusScope, https://demo.consensusscope.cn/, https://youtu.be/6reDMnZGGW4, and https://api.consensusscope.cn/docs. The hosted demo is served through a Cloudflare named tunnel to a self-hosted Streamlit and FastAPI deployment rather than Streamlit Community Cloud, which avoids cold-start failures during review. The repository contains installation instructions, a one-command Streamlit launch script, backend startup instructions, and scripts/run_public_gec_benchmark.py for reproducing the aggregate public benchmark. JFLEG can be downloaded automatically by the script; CoNLL-2014, FCE, and W&I+LOCNESS are retrieved from their official releases. Raw corpora and per-item derived outputs are not committed; aggregate result tables are included for inspection.

10 Ethics and Limitations

ConsensusScope is a teacher-support tool, not an automatic essay scorer or teacher replacement. Users should upload only anonymized student writing; names, student IDs, emails, class identifiers, and other personally identifiable information should be removed before use. The current benchmark validates graph-backed review-routing behavior with public correction labels and constructed distractors. It does not measure student learning, teacher satisfaction, time-on-task, or real LLM feedback quality. The two-teacher pilot covers 30 feedback items and should be read as a di-

Case	AI feedback pattern	Graph route	Diagnostic value
FB-002	Local phrase improvement	local edit → auto-accept	Both teachers rate it safe
FB-003	Reverses the student's thesis	meaning preservation → review	Blocks meaning change
FB-008	Adds unsupported exam-score claim	content grounding → review	Blocks unsupported evidence
FB-005	"can make" changed to "makes"	meaning preservation → review	Reveals wrong local grammar correction

Table 4: Representative cases from the teacher diagnostic pilot. The graph route is constructed from deploy-time signals; teacher ratings are used only for offline analysis.

Policy	Auto share	Auto acc.	Review share	Errors reviewed
Accept low; review med/high	0.329	1.000	0.671	1.000
Accept low/med; review high	0.336	1.000	0.664	1.000
Review all	0.000	–	1.000	1.000

Table 5: Pooled review-routing utility over the public learner-corpus benchmark. Errors reviewed is the fraction of observed incorrect or unsafe feedback candidates routed to the human-review queue.

agnostic check, not a classroom user study. Future work should expand the teacher sample, add authentic anonymized classroom data, and keep deploy-time graph signals separate from offline diagnostic ratings.

11 Conclusion

ConsensusScope demonstrates Feedback Safety Graph Routing for AI-generated ESL writing feedback. By defining feedback-level safety routing, extracting deploy-time risk signals, constructing item-level safety graphs, and routing uncertain or unsafe suggestions into a teacher queue, the system turns AI feedback from a final rewrite into an auditable teacher-in-the-loop decision.

References

Christopher Bryant, Mariano Felice, Øistein E. Andersen, and Ted Briscoe. 2019. The BEA-2019 shared task on grammatical error correction. In Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications, pages 52–75, Florence, Italy. Association for Computational Linguistics.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG evaluation using GPT-4 with better human alignment. arXiv preprint arXiv:2303.16634.

Nitin Madnani, Jill Burstein, Norbert Elliot, Beata Beigman Klebanov, Diane Napolitano, Slava Andreyev, and Maxwell Schwartz. 2018. Writing mentor: Self-regulated writing feedback for struggling writers. In Proceedings of the 27th International Conference on Computational Linguistics: System Demonstrations, pages 113–117, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Courtney Napoles, Keisuke Sakaguchi, and Joel Tetreault. 2017. JFLEG: A fluency corpus and benchmark for grammatical error correction. In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers, pages 229–234, Valencia, Spain. Association for Computational Linguistics.

Hwee Tou Ng, Siew Mei Wu, Ted Briscoe, Christian Hadiwinoto, Raymond Hendy Susanto, and Christopher Bryant. 2014. The CoNLL-2014 shared task on grammatical error correction. In Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task, pages 1–14, Baltimore, Maryland. Association for Computational Linguistics.

Jim Ranalli. 2018. Automated written corrective feedback: how well can students make use of it? Computer Assisted Language Learning, 31(7):653–674.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations.

Helen Yannakoudakis, Ted Briscoe, and Ben Medlock. 2011. A new dataset and method for automatically grading ESOL texts. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pages 180–189, Portland, Oregon, USA. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In Advances in Neural Information Processing Systems.